

Lab 5

METHOD

For this task, since I knew it had to be done in linear time, my first thought to sort and enumerate that way were out of the picture because sorting is $O(n \log(n))$. So then I thought about using a different kind of sorting.

APPROACH

For this alternate sorting I created another array which would keep track of the number of papers that had exactly n citations. This was implemented so $\text{frequency}[n] = \text{number of papers with exactly } n \text{ citations}$. Then once they were “sorted” I could simply iterate from the end and keep a running total, and then once $\text{total} \geq n$, I could return n , since n would satisfy the conditions of the h-index.

REASONING

This works in linear time because creating the frequency array, populating it, and checking for h-index all run in linear time, so the program itself will be linear. The algorithm works by rearranging the given data into a more manageable form.

COMPLEXITY

This algorithm works in linear time

REFLECTION

I used these two test cases:

```
print("Test 1")
citations = [3, 0, 6, 1, 5]
print(citations)
print(f"The h-index for this set is: {h_index(citations)}")
```

```
print("Test 2")
citations = [1, 3, 1]
print(citations)
print(f"The h-index for this set is: {h_index(citations)}")
```

I chose to use these two cases because since they were given as examples in the Lab Document I was able to use them to quickly verify correctness.